Work Timer – licensed to ‘farwaasim’

Technical Report

1. Overview

Work Timer is a small always-on-top desktop gadget for Windows that tracks active working hours and idle time, persists state across runs, and maintains a per-day log of usage. The application is implemented in Python using Tkinter for the GUI and pystray for system tray integration. State persists to %APPDATA%\WorkTimer\ as JSON.

This report documents the system design, the issues encountered during development, and the resolution of each.

2. System Architecture

2.1 Components

Component	Responsibility
WorkTimerApp	Application controller; owns Tkinter root and tray icon
PillCanvas	Custom canvas widget rendering the gradient pill with centered timer text
State persistence	load_state / save_state for current session; load_log / save_log for daily history
Idle detection	Win32 GetLastInputInfo accessed via ctypes
Tray icon	pystray.Icon on a daemon thread, marshalling actions to the Tk thread via root.after

2.2 State Model

Two JSON files are stored in %APPDATA%\WorkTimer\:

- timer_state.json: current session accumulated work seconds, accumulated idle seconds, and last window position.
- daily_log.json: dictionary keyed by ISO date (YYYY-MM-DD), each value containing work_seconds and idle_seconds.

State is written on every tick while running, on every Start, Pause, and Reset transition, and on application exit.

2.3 Timing Model

A monotonic clock anchor (time.monotonic()) is captured on Start. The current work total is computed as saved_work + (now - run_started_at), ensuring resilience against system clock changes. The 1-second tick loop computes a delta against the previous tick and attributes that delta to the work counter unconditionally, and to the idle counter when system idle time exceeds the threshold (180 seconds).

2.4 UI States

- Pill (collapsed): 150x44 px frameless window with a magenta-keyed transparent background, displaying a horizontal lavender-to-cream gradient capsule with centered timer text.
- Panel (expanded): cream card with separate Work and Idle counters and Start, Pause, Reset, View Log, Collapse, and Quit controls.
- Tray icon: present at all times; provides Show/Hide pill, Start/Pause, View Log, Reset, and Quit.

3. Issues Encountered and Resolutions

3.1 Pill Not Draggable from Inner Padding

Symptom: Click-and-drag on the pill failed to move the window when the click landed on certain pixels.

Root cause: Drag event handlers (ButtonPress-1, B1-Motion, ButtonRelease-1) were bound only to the outer frame and the timer label widget. Padding pixels inside the frame had no widget owning them, so click events on those pixels did not trigger any handler.

Resolution: Replaced the frame-plus-label structure with a single Canvas widget covering the full pill area. Drag handlers were also bound to all panel widgets so the expanded view is draggable from any pixel. A 3-pixel movement threshold was added before a press is interpreted as a drag, so light clicks still register as taps to toggle the panel.

3.2 State Files Created Inside PyInstaller Build Output

Symptom: After running the executable from dist\WorkTimer.exe for testing, timer_state.json and daily_log.json were written into the dist folder, blocking deletion of the build artifact.

Root cause: The original implementation computed the state file path relative to os.path.dirname(os.path.abspath(sys.argv[0])), which resolves to the directory of the running executable. Running the executable from dist therefore wrote state into dist.

Resolution: State paths were moved to %APPDATA%\WorkTimer\ using os.environ.get("APPDATA"). A one-time migration routine runs on startup: if legacy state files exist next to the executable and no state exists at the new location, they are moved (not copied) to %APPDATA%. This decouples user data from executable location.

3.3 Icon Illegible at Small Sizes

Symptom: The first icon design (peach hourglass with bow, sand, sparkles, and detail elements) rendered correctly at 256x256 but was unreadable at 32x32, the size used by the Windows taskbar and File Explorer.

Root cause: Excessive detail density. Multiple small features (sparkles, blush dots, ribbon highlights, hourglass frame plates) competed for the limited pixel budget at small sizes.

Resolution: Redesigned with one primary shape (a chunky cat face) and a two-color palette (lavender and cream). Stroke widths were chosen so they remain visible at 16-pixel rendering. Verification was performed by rendering the SVG at 32, 64, and 128 px and inspecting all three at actual pixel size before accepting the design.

3.4 Window Did Not Appear in Taskbar

Symptom: When the application was running, no entry appeared in the Windows taskbar at the bottom of the screen, regardless of icon configuration.

Root cause: Not a defect. root.overrideredirect(True) is required to produce a frameless window suitable for a gadget aesthetic. A documented side effect on Windows is that frameless top-level windows are excluded from the taskbar.

Resolution: Added a system tray icon via pystray. The tray icon provides the persistent visual presence and menu access that a taskbar entry would have provided in a non-frameless application. The tray icon runs on a daemon thread; menu actions are marshalled back to the Tk main thread using root.after(0, callback) to avoid cross-thread Tk calls.

3.5 Stale Icon on Executable in File Explorer

Symptom: After rebuilding the executable with an updated icon.ico, File Explorer continued to display the previous icon on the executable, while the running application tray icon correctly showed the new design.

Diagnostic step: File timestamps confirmed icon.ico (last modified 12:42 AM) preceded the executable build (1:00 AM) by 18 minutes, ruling out a stale build.

Root cause: Windows icon cache. Windows caches icons by file path; once cached, subsequent changes to the file embedded icon are not reflected until the cache is invalidated.

Resolution: Restarting explorer.exe via Task Manager forced a cache refresh and the correct icon appeared. Renaming the file and renaming it back is an alternative method that also invalidates the cached entry.

3.6 Smart App Control Blocks the Built Executable

Symptom: On Windows 11, attempting to run the PyInstaller-built WorkTimer.exe produced a Smart App Control dialog stating that the application could not be confirmed and was blocked. The dialog provided no override button.

Root cause: Smart App Control is a Windows 11 feature distinct from SmartScreen. Unlike SmartScreen, which provides a Run Anyway override via the More Info link, Smart App Control has no per-application override. It blocks all unsigned executables when enabled. Disabling Smart App Control is a one-way operation; it cannot be re-enabled without resetting Windows.

Resolution: Distribution model changed from a packaged executable to a Python script with a launcher batch file. WorkTimer.bat invokes pythonw work_timer.py (the windowed Python interpreter, which suppresses the console window). A Desktop shortcut points to the batch file, with its icon set to icon.ico via the shortcut Properties dialog. Smart App Control does not block the Python interpreter itself, so this

approach runs without policy intervention while preserving the user-facing behavior of a double-clickable installed application.

3.7 Batch File Created with Wrong Extension

Symptom: Creating WorkTimer.bat via Explorer New > Text Document and renaming produced a file that remained WorkTimer.bat.txt.

Root cause: Default Windows configuration hides known file extensions. The original file was New Text Document.txt; renaming the visible portion to WorkTimer.bat left the hidden .txt extension intact, producing WorkTimer.bat.txt.

Resolution: Created the file via PowerShell using a here-string and Out-File -Encoding ASCII, which writes the file with the literal name WorkTimer.bat and bypasses Explorer extension handling entirely.

4. Final Configuration

4.1 Runtime Files

File	Purpose
work_timer.py	Application source
icon.ico	Multi-resolution icon (16, 24, 32, 48, 64, 128, 256 px)
WorkTimer.bat	Launcher invoking pythonw
Desktop shortcut to WorkTimer.bat	User-facing entry point with custom icon

4.2 Persistent Data

Located at %APPDATA%\WorkTimer\:

- timer_state.json
- daily_log.json

4.3 Dependencies

- Python 3.8 or later with Tkinter
- pystray
- Pillow

4.4 Platform

Windows 10 and Windows 11. Idle detection is implemented using Windows-specific Win32 calls and is not portable to other operating systems without adaptation.

5. Known Limitations

1. System-wide idle detection: GetLastInputInfo reports input across the entire system. Periods of consuming media without keyboard or mouse input (video playback, reading) are counted as idle.
2. Midnight rollover precision: A timer tick that crosses midnight is attributed to the day in which the tick is processed. Maximum skew is one tick interval (1 second).
3. No per-session log: Only daily totals are recorded. Individual Start and Pause cycles are not retained.
4. Reset is destructive: Reset clears current session counters with confirmation, but daily log entries cannot be edited or deleted from within the application.
5. Code signing not performed: The application is distributed as source plus a launcher. Packaging as a signed executable would require a code signing certificate.

6. Future Work

- Optional prompt on return from idle to retain or discard the idle interval.
- Aggregated weekly and monthly summaries.
- Per-session annotations.
- Cross-platform idle detection abstraction to support macOS and Linux.